L.V. Causality Marketing Assignment 3

April 6, 2026

Contents

1 Import Data **2**

2 Regression Adjustment **2**

3 Matching Comparisons **2**
3.1 1:1 Nearest Neighbor Matching WITHOUT Replacement 2
3.2 1:1 Nearest Neighbor Matching WITH Replacement 3
3.3 Euclidean Distance Matching 3
3.4 Comparing Matching Quality Across Methods 4

4 Inverse Probability Weighting (IPW) **4**
4.1 IPW Critical Discussion Points 6

5 Augmented Inverse Probability Weighting (AIPW) **6**
5.1 AIPW Critical Discussion Points 7

6 Double Machine Learning (DML): PLR with Cross-Fitting **7**
6.1 DML Discussion Points 8

7 Comparison of Treatment Effects Across Methods **8**

8 Discussion **8**
8.1 Comparison of ATE vs. ATT Estimates 9
8.2 Methodological Divergence and Instability 9
8.3 The Role of Double Robustness and Machine Learning 9
8.4 Conclusion: Which Estimate to Trust? 9
8.5 Suggestions for the Marketing Manager 10

9 Reflection on AI Use **10**

1 Import Data

```
casedata_task3 <- read.csv("CaseData_Task3.csv", stringsAsFactors =
    FALSE)
str(casedata_task3)
head(casedata_task3)
```

2 Regression Adjustment

The estimate for the discount coefficient is the Regression Adjustment ATE.

```
model_adj <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10
    +
                  X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 +
                     X20,
                  data = casedata_task3)
summary(model_adj)
```

We extract the ATE value and calculate robust standard errors using the HC3 estimator, which is more reliable in small samples and with heteroskedasticity.

```
coeftest(model_adj, vcov = vcovHC(model_adj, type = "HC3"))
ate <- coef(model_adj)["d"]
cat("The estimated ATE for the treatment is:", ate, "\n")
```

3 Matching Comparisons

Preparing the data for matching exercises:

```
ps_formula <- d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
                  X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 +
                     X20
```

3.1 1:1 Nearest Neighbor Matching WITHOUT Replacement

```
match_no_replace <- matchit(ps_formula,
                            data = casedata_task3,
                            method = "nearest",
                            distance = "glm",
                            replace = FALSE,
                            ratio = 1)

cat("Balance Summary (Without Replacement):\n")
print(summary(match_no_replace, un = FALSE))

data_matched_no_replace <- match.data(match_no_replace)

model_no_replace <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 +
                        X9 + X10 + X11 + X12 + X13 + X14 + X15 + X16 +
                        X17 + X18 + X19 + X20,
                        data = data_matched_no_replace,
                        weights = weights)
```

Looking at the balance table, the results are not great. Many covariates still have SMDs well above 0.1: X12 is at −1.54, X11 at −0.83, X13 at −0.93. The propensity score distance SMD is 1.92, which is quite high. So even after matching, the treated and control groups still look pretty different on these variables. We think this happens because without replacement, once a good control unit gets matched it's gone, and later treated units get stuck with worse matches. This makes us less confident in the ATT from this specification.

We extract the ATT value and calculate robust standard errors using HC estimation to account for the matched data structure and potential heteroscedasticity.

```
print(coeftest(model_no_replace, vcov. = vcovHC))
att <- coef(model_no_replace)["d"]
cat("The estimated ATT is:", att, "\n")
```

3.2 1:1 Nearest Neighbor Matching WITH Replacement

We use `get_matches()` instead of `match.data()` when matching with replacement to properly duplicate rows that are used as matches multiple times.

```
match_with_replace <- matchit(ps_formula,
                              data = casedata_task3,
                              method = "nearest",
                              distance = "glm",
                              replace = TRUE,
                              ratio = 1)

cat("Balance Summary (With Replacement):\n")
print(summary(match_with_replace, un = FALSE))

data_matched_with_replace <- get_matches(match_with_replace)

model_with_replace <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 +
                         X9 + X10 + X11 + X12 + X13 + X14 + X15 + X16 +
                         X17 + X18 + X19 + X20,
                         data = data_matched_with_replace,
                         weights = weights)
```

Balance is much better here. Most SMDs drop below 0.13, and the propensity score distance SMD is basically 0 (0.0005). X7 and X8 are still a bit above 0.1 (∼0.22 to 0.24), but overall this is a big improvement over the without-replacement version. This makes sense, since allowing replacement means each treated unit can grab the best available control, even if that control has already been used. The downside is that some controls get reused a lot, which is why we need to cluster the standard errors by subclass below.

```
print(coeftest(model_with_replace, vcov. = vcovCL, cluster = ~subclass))
att <- coef(model_with_replace)["d"]
cat("The estimated ATT is:", att, "\n")
```

3.3 Euclidean Distance Matching

```
euc_out <- matchit(
  d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
       X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
```

```
  data = casedata_task3,
  method = "nearest",
  distance = "euclidean",
  normalize = TRUE
)

summary(euc_out)
m_data_euc <- match.data(euc_out)
lm_euc <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
             X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
             data = m_data_euc, weights = weights)

summary(lm_euc)
cat("\nRobust Standard Errors (Euclidean Matching):\n")
print(coeftest(lm_euc, vcov. = vcovHC(lm_euc, type = "HC3")))
```

3.4 Comparing Matching Quality Across Methods

Looking at the balance tables side by side, Euclidean distance matching and PSM without replacement perform similarly, and both are pretty bad. The Euclidean SMDs are actually slightly worse on the key problem covariates (X12 = −1.60 vs. −1.54 for PSM, X11 = −0.86 vs. −0.83). Neither method manages to bring X10 to X15 anywhere close to the 0.1 threshold. PSM with replacement is clearly the best of the three, it gets most covariates under 0.13 and the propensity score distance is essentially perfectly balanced. So in terms of matching quality, with-replacement PSM wins by a wide margin here, though it comes at the cost of reusing control units and needing clustered standard errors. The poor balance in the other two methods means their ATT estimates should probably be taken with a grain of salt.

On common support: the sample sizes tables show that 64 control units (out of 2532) went unmatched, meaning they had no reasonably close treated counterpart. No treated units were discarded. So we're not losing a huge chunk of the data, which is reassuring, but the fact that those 64 controls couldn't find a match does hint at the limited overlap between the groups, which is also visible in the propensity score density plot in Figure 1. The separation in that plot explains why both the matching balance and the IPW weights struggle here.

4 Inverse Probability Weighting (IPW)

```
ipw_vars <- c("y", "d", paste0("X", 1:20))
df_ipw <- casedata_task3[complete.cases(casedata_task3[, ipw_vars]), ]

ps_model <- glm(
  d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
      X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
  data = df_ipw,
  family = binomial(link = "logit")
)

df_ipw$ps <- predict(ps_model, type = "response")
```

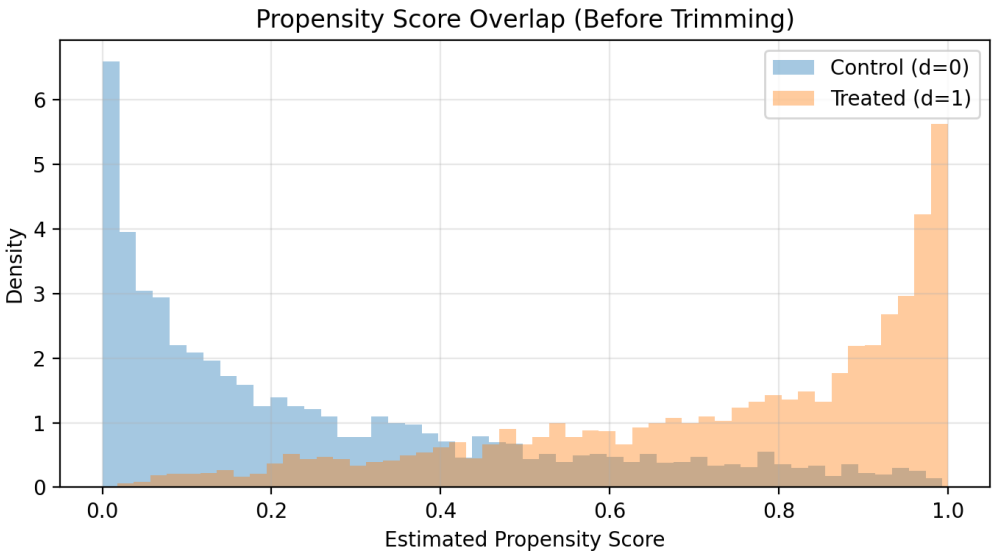

Figure 1: Propensity score distributions by treatment group before trimming. The limited overlap between the two distributions is the root cause of the unstable IPW estimate.

```
df_ipw$ps_trim <- pmin(pmax(df_ipw$ps, 0.01), 0.99)

estimate_ipw_ate <- function(data, trim = 0.01) {
  ps_fit <- glm(
    d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
      X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
    data = data,
    family = binomial(link = "logit")
  )
  ps_hat <- predict(ps_fit, newdata = data, type = "response")
  ps_hat <- pmin(pmax(ps_hat, trim), 1 - trim)
  mean(data$y * (data$d / ps_hat - (1 - data$d) / (1 - ps_hat)))
}

ate_ipw_manual <- estimate_ipw_ate(df_ipw, trim = 0.01)
cat("\nManual IPW ATE (primary estimate):", round(ate_ipw_manual, 4), "\
    n")

# Bootstrap inference
set.seed(123)
B_ipw <- 1000
n_ipw <- nrow(df_ipw)
boot_ipw_ate <- numeric(B_ipw)

for (b in seq_len(B_ipw)) {
  idx <- sample.int(n_ipw, size = n_ipw, replace = TRUE)
  boot_data <- df_ipw[idx, ]
  boot_ipw_ate[b] <- estimate_ipw_ate(boot_data, trim = 0.01)
}

ipw_boot_mean <- mean(boot_ipw_ate)
ipw_boot_se <- sd(boot_ipw_ate)
ipw_boot_ci <- quantile(boot_ipw_ate, probs = c(0.025, 0.975))

# Secondary: IPW weighted regression
```

```r
df_ipw$ipw_weight <- ifelse(df_ipw$d == 1,
                            1 / df_ipw$ps_trim,
                            1 / (1 - df_ipw$ps_trim))

lm_ipw <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
             X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
             data = df_ipw, weights = ipw_weight)
print(coeftest(lm_ipw, vcov. = vcovHC(lm_ipw, type = "HC3")))
```

4.1 IPW Critical Discussion Points

The manual IPW estimate came out at around -318, which obviously doesn't make sense for a sales effect. Looking at the overlap plot in Figure 1, I think the issue is clear: the propensity score distributions for treated and control are quite separated. A lot of control units have scores close to 0, and when you invert those (dividing by something like 0.01 gives a weight of $\sim$100), a few observations end up dominating the whole estimate. The 0.01/0.99 trimming we used wasn't aggressive enough to fix this.

This is basically the classic problem with IPW: it's very sensitive to poor overlap. The weighted regression version ($\sim$15.81) does much better because the regression component doesn't rely purely on the weights. It also shows why AIPW and DML are generally more appealing: they don't put all their eggs in the weighting basket.

5 Augmented Inverse Probability Weighting (AIPW)

This section implements AIPW manually using the doubly robust score:

$$\widehat{\text{ATE}} = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{T_i}{\hat{p}(X_i)}(Y_i - \hat{\mu}_{T=1}(X_i)) + \hat{\mu}_{T=1}(X_i)\right) - \frac{1}{N}\sum_{i=1}^{N}\left(\frac{(1-T_i)}{1-\hat{p}(X_i)}(Y_i - \hat{\mu}_{T=0}(X_i)) + \hat{\mu}_{T=0}(X_i)\right)$$

```r
df <- casedata_task3
df$treatment <- df$d
df$sales <- df$y
covariates <- paste0("X", 1:20)

aipw_vars <- c("treatment", "sales", covariates)
df_aipw <- df[complete.cases(df[, aipw_vars]), aipw_vars]

estimate_aipw_ate <- function(data, covariate_names, trim = 0.01) {
  ps_formula <- as.formula(
    paste("treatment ~", paste(covariate_names, collapse = " + "))
  )
  ps_model <- glm(ps_formula, data = data, family = binomial(link = "
      logit"))
  p_hat <- predict(ps_model, newdata = data, type = "response")
  p_hat <- pmin(pmax(p_hat, trim), 1 - trim)

  mu1_formula <- as.formula(
    paste("sales ~", paste(covariate_names, collapse = " + "))
```

```
  )
  outcome_model_treated <- lm(mu1_formula, data = data[data$treatment ==
      1, ])
  outcome_model_control <- lm(mu1_formula, data = data[data$treatment ==
      0, ])

  mu1_hat <- predict(outcome_model_treated, newdata = data)
  mu0_hat <- predict(outcome_model_control, newdata = data)

  score_treated <- (data$treatment / p_hat) * (data$sales - mu1_hat) +
     mu1_hat
  score_control <- ((1 - data$treatment) / (1 - p_hat)) *
                   (data$sales - mu0_hat) + mu0_hat
  mean(score_treated) - mean(score_control)
}

aipw_ate <- estimate_aipw_ate(df_aipw, covariates, trim = 0.01)
cat("Manual AIPW ATE estimate:", round(aipw_ate, 4), "\n")

# Bootstrap inference
set.seed(123)
B <- 1000; n <- nrow(df_aipw); boot_ate <- numeric(B)
for (b in seq_len(B)) {
  idx <- sample.int(n, size = n, replace = TRUE)
  boot_ate[b] <- estimate_aipw_ate(df_aipw[idx, ], covariates, trim =
     0.01)
}
aipw_se <- sd(boot_ate)
aipw_ci <- quantile(boot_ate, probs = c(0.025, 0.975))
```

5.1 AIPW Critical Discussion Points

- Doubly robust consistency: AIPW is consistent if either the propensity score model or the outcome regression models are correctly specified.
- Efficiency: Compared with simple IPW, AIPW is typically more stable and can achieve the semiparametric efficiency bound under correct specification.
- Debiasing logic: The residual-weighted terms $(Y - \hat{\mu})$ correct the regression predictions using inverse-probability weighting.
- Overlap/common support: Propensity scores should stay away from 0 and 1; weak overlap creates extreme weights and unstable estimates, so diagnostics and trimming are important.

6 Double Machine Learning (DML): PLR with Cross-Fitting

We estimate the ATE with DoubleML using the partially linear regression model:

$$Y = \beta D + g(X) + \epsilon, \qquad D = m(X) + \xi$$

Cross-fitting (sample splitting) is used to reduce overfitting bias in nuisance estimation, and the orthogonal score makes the final β less sensitive to small nuisance-model errors.

```
x_cols <- paste0("X", 1:20)
dml_vars <- c("y", "d", x_cols)
dml_df <- casedata_task3[complete.cases(casedata_task3[, dml_vars]), dml
    _vars]

dml_data <- DoubleMLData$new(data = dml_df, y_col = "y",
                             d_cols = "d", x_cols = x_cols)

# Run DML with three learner families: Lasso, Random Forest, XGBoost
# (full helper functions omitted for brevity (see .qmd for details))

dml_lasso <- run_dml_spec("Lasso", "regr.glmnet", "classif.lasso")
dml_rf    <- run_dml_spec("Random Forest", "regr.ranger", "classif.
    ranger")
dml_xgb   <- run_dml_spec("XGBoost", "regr.xgboost", "classif.xgboost")

dml_results <- rbind(dml_lasso, dml_rf, dml_xgb)
```

6.1 DML Discussion Points

- DML solves regularization bias and overfitting bias: sample splitting and cross-fitting reduce overfitting from flexible learners.
- Predictive performance of nuisance models is central: lower RMSE/log-loss means cleaner residualization (orthogonalization) for treatment-effect estimation.
- Compared with weighting estimators, PLR-style partialling out avoids direct division by propensity scores, which can be more stable under weak overlap.
- Even with ML-based nuisance models, causal validity still depends on conditional independence and excluding bad controls (e.g., colliders).

7 Comparison of Treatment Effects Across Methods

```
# (Comparison table construction code (see .qmd for full details))
comparison_table <- rbind(comparison_table_base, dml_comparison)
print(comparison_table, row.names = FALSE)
```

Note: ATT rows are matching estimands; all other rows report ATE. DML rows include learner-specific PLR estimates with 5-fold cross-fitting and orthogonal-score inference.

8 Discussion

One thing worth noting upfront is that every method used here relies on the same assumption: that the 20 covariates are enough to account for all the confounding between who gets the discount and how much they buy. We can't really test this. If there's something unobserved driving both treatment and sales (e.g., customer motivation), all of these estimates could be off. But given that we have 20 covariates covering demographics and purchasing behavior, it seems like a reasonable assumption for this dataset.

8.1 Comparison of ATE vs. ATT Estimates

An important thing to keep in mind when comparing the results is that not all methods estimate the same thing.

- **ATE methods** (Regression, IPW, AIPW, DML) estimate the effect if the discount were given to everyone. Ignoring the broken manual IPW, these range from roughly 13.70 to 18.85.
- **ATT methods** (Matching) estimate the effect for the customers who actually got the discount, ranging from about 16.16 to 23.25.

The ATT estimates being higher than the ATE could suggest that the customers who received the discount also happen to respond more strongly to it, something like selection on returns. Though we're not 100% sure this isn't partly driven by differences in how the methods handle the data (e.g., the poor balance in the without-replacement matching might inflate things).

8.2 Methodological Divergence and Instability

IPW blowup: The manual IPW gave -318, which is nonsense. we already discussed this above. It's an overlap problem with extreme weights. The weighted regression IPW ($\sim$15.81) is much more reasonable since it doesn't rely only on the weights.

Matching differences: The without-replacement estimate (17.15) and with-replacement estimate (23.25) are pretty far apart. Given the balance diagnostics, we'd trust the with-replacement version more, the without-replacement matching left large SMDs on X10 to X14, so those matched pairs weren't really comparable. The with-replacement version got balance mostly under control, though it comes with more variance from reusing controls.

8.3 The Role of Double Robustness and Machine Learning

AIPW (18.85) is doubly robust, meaning it stays consistent if either the propensity score model or the outcome model is right. In theory it should also be more efficient than plain IPW.

DML results vary by learner:

- XGBoost: 18.35
- Lasso: 15.23
- Random Forest: 13.70

The fact that XGBoost and AIPW land in a similar range ($\sim$18) is interesting. It might mean there are some non-linearities in the covariate-outcome relationship that OLS misses but these methods pick up. The Random Forest estimate being lower is a bit puzzling and could reflect differences in how well the nuisance models fit.

8.4 Conclusion: Which Estimate to Trust?

We'd lean toward trusting the **AIPW** or **DML-XGBoost** estimates, for a few reasons:

1. They don't assume linearity the way plain regression does.

2. AIPW has the double robustness property, which gives some insurance against model misspecification.
3. They use the full dataset rather than discarding unmatched units, so they're answering the ATE question more directly.

That said, none of these methods can fully overcome the unconfoundedness assumption. If there are important unobserved confounders, all estimates could be biased.

8.5 Suggestions for the Marketing Manager

Across all the credible methods, the discount seems to increase sales by roughly $15 to 19 per customer (ATE). The campaign appears to work. Whether it's worth expanding depends on the cost of the discount itself. If the discount costs less than $15 per customer, broader rollout looks profitable.

For the customers currently being targeted, the effect might be even larger (∼$17 to 23 based on matching ATT). So the existing targeting seems to be picking customers who respond well.

If the company wants to be more confident before scaling up, running a proper randomized experiment on a sample of untargeted customers would help. That would give a clean ATE estimate without having to worry about whether the covariates capture everything. For now, the observational evidence is encouraging but comes with the usual caveat that we're assuming the data accounts for all relevant confounders.

9 Reflection on AI Use

We used AI tools at various points during this assignment, mainly in a supporting role rather than for generating the analysis or writing from scratch.

Tools used: Primarily Google Gemini for code-related tasks, and NotebookLM for revisiting theoretical concepts from the course material.

What we used AI for:

- Debugging code when we ran into errors or unexpected output, and getting suggestions on how to improve code quality (e.g., fixing syntax issues, making sure we were using the right functions from MatchIt or DoubleML).
- Checking for oversights in our analysis, essentially using it as a second pair of eyes to see if we missed something obvious or made a methodological mistake.
- Searching for information to refresh our understanding of the theory behind the methods, particularly around IPW, AIPW, and how DML cross-fitting works.
- NotebookLM was useful for going back over course readings and lecture material to make sure we had the concepts right before writing up the discussion.

How we handled AI input: We treated AI suggestions as starting points rather than final answers. When it flagged something or suggested a change, I checked it against the course material and our own understanding before incorporating it. The analytical choices, interpretations, and written discussion are our own.